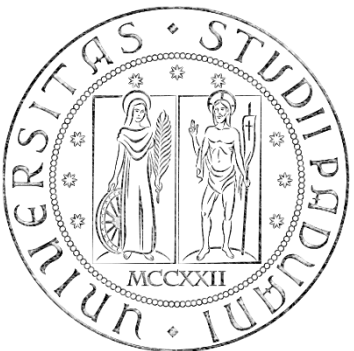


Biomedical Wearable Technologies
for Healthcare and Wellbeing

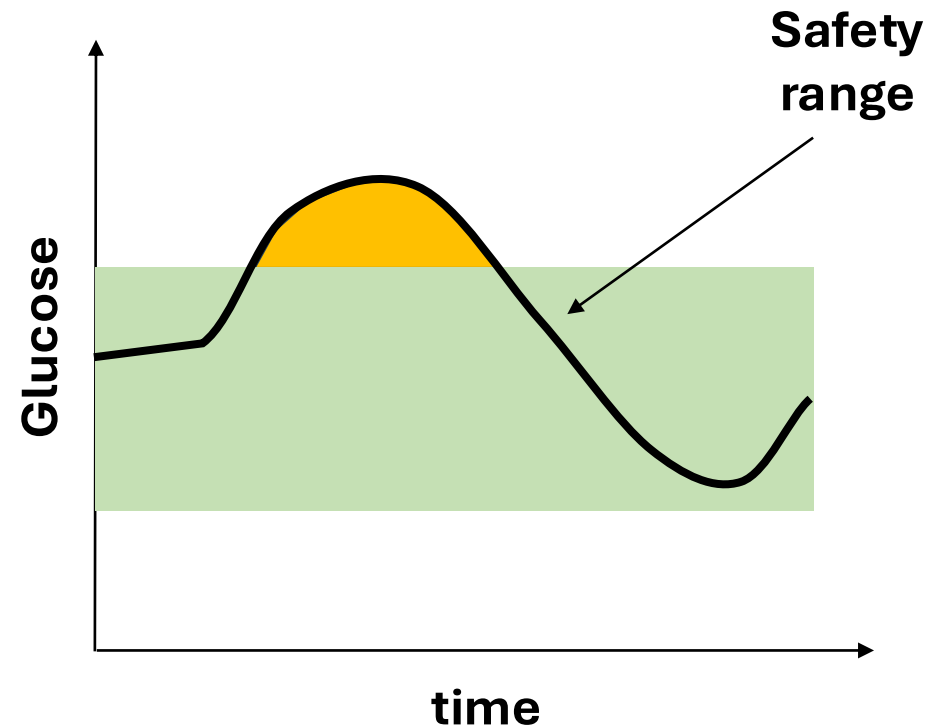
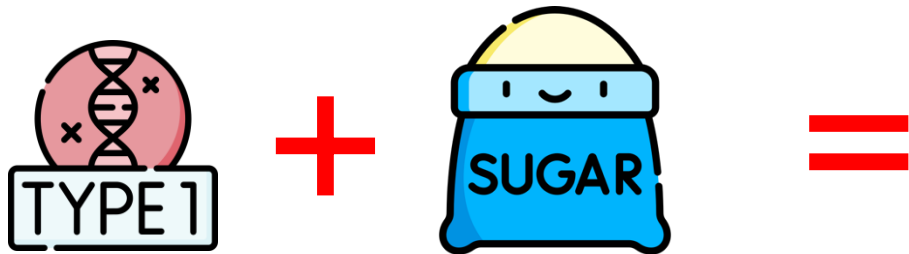
Integrating AI Models in Flutter – Part 1

A.Y. 2025-2026
Giacomo Cappon



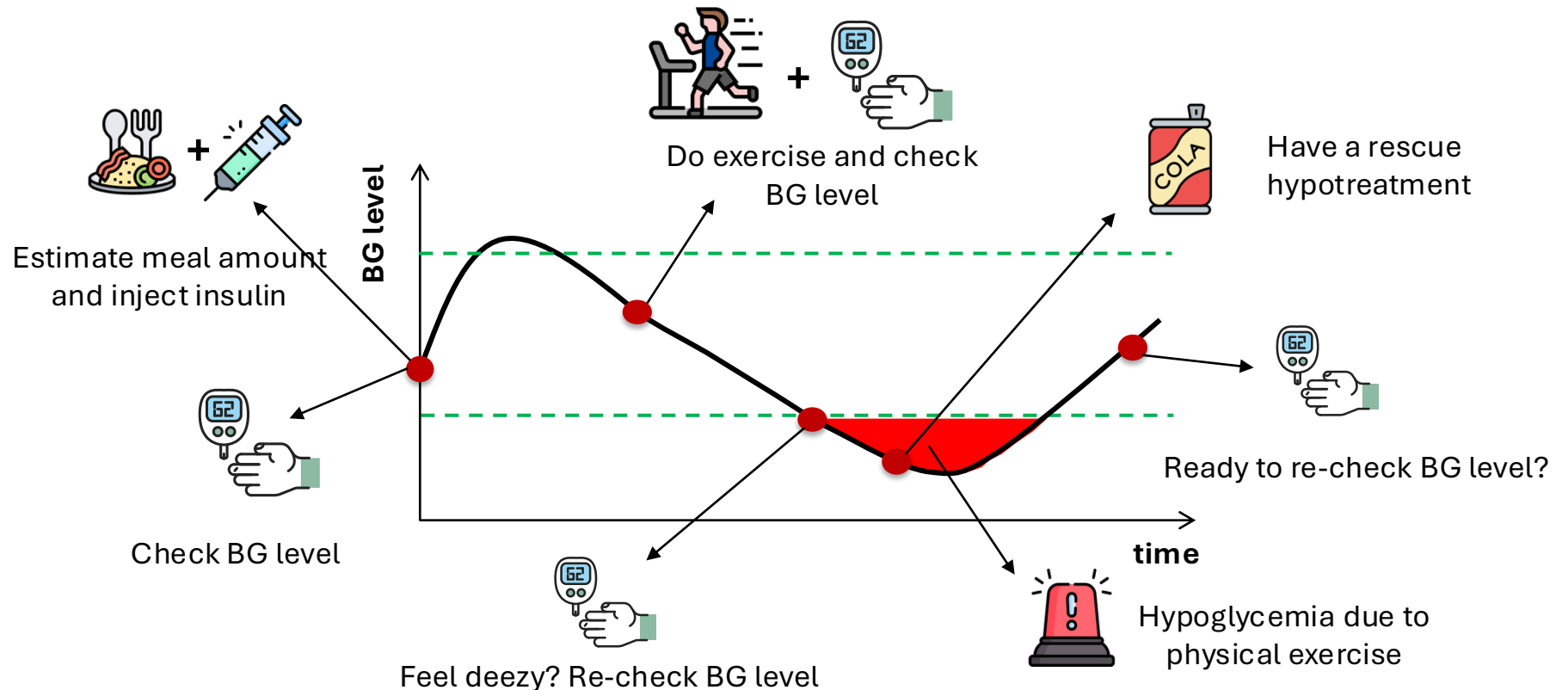
Type 1 Diabetes

T1D is a chronic, autoimmune pathology, in which the **pancreas is no longer able to produce insulin**, which must then be externally injected



The burden of T1D management

T1D management is a constant learning process about how daily conditions/routine affect blood glucose (BG) levels.



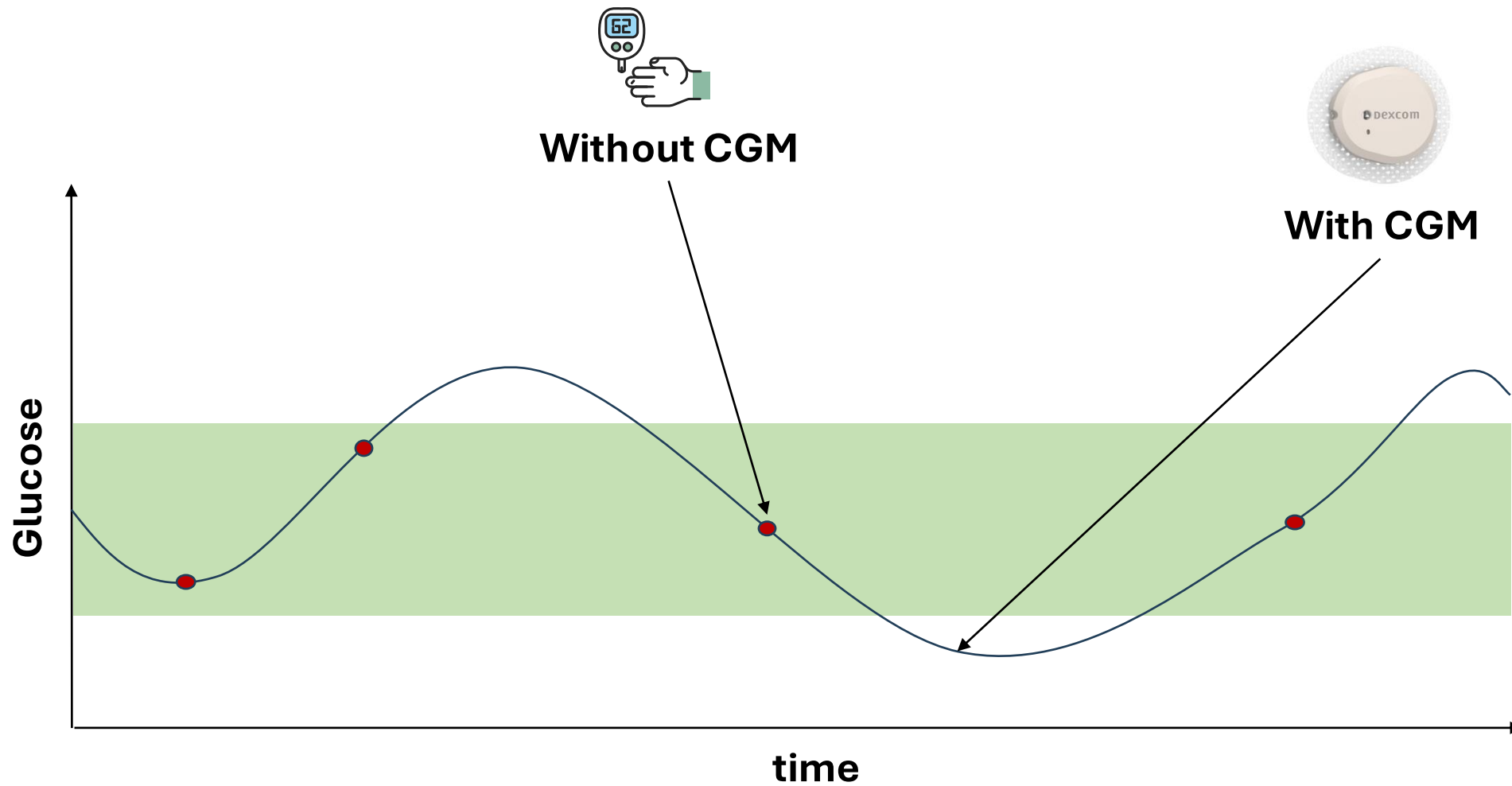
The role of wearable devices in T1D

Continuous Glucose Monitoring (CGM) systems:

- Minimally-invasive sensors that measure interstitial glucose every $\sim 1/5$ minutes
- Micro-needle under the skin that leverages glucose-oxidase reaction
- Surpasses the traditional fingerprick devices



What does it mean in practice?



What does it mean in practice?



CGM



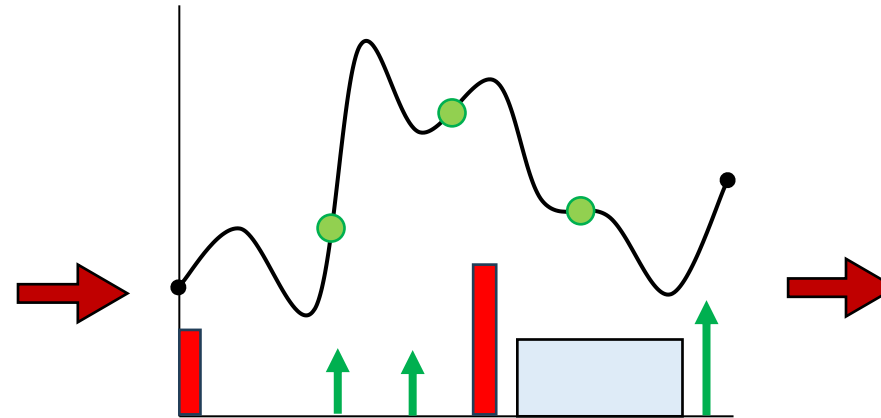
Apps for meal
intake estimation



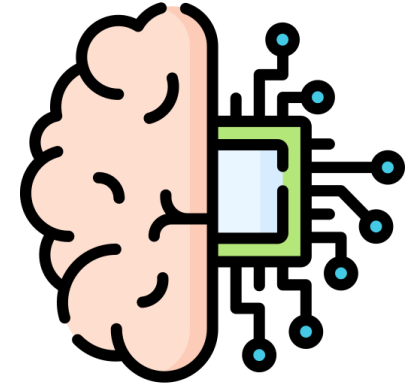
Smart insulin pen
Insulin pump



Activity tracker



Rich patient-generated
dataset

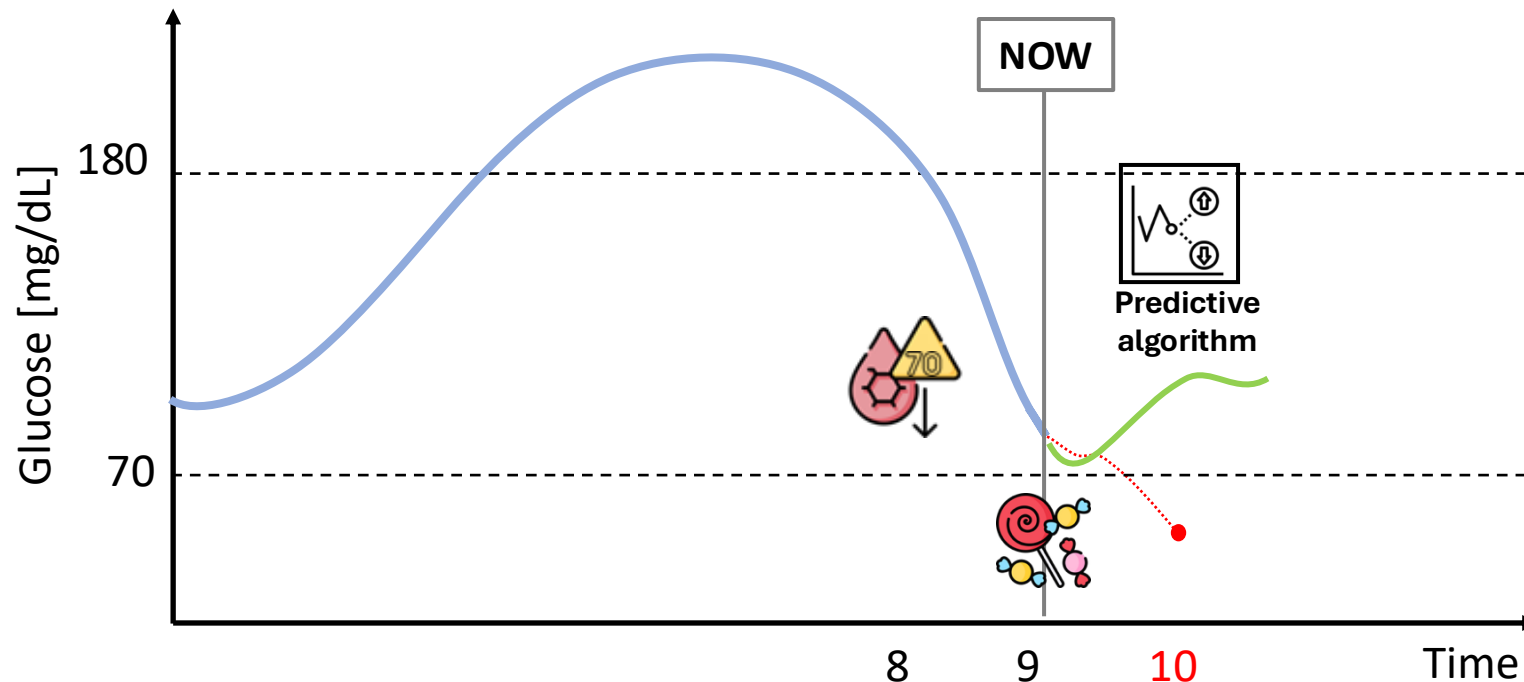


AI-based algos

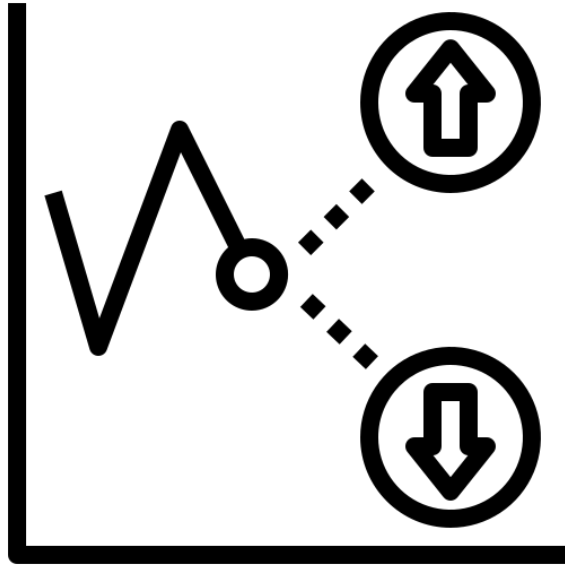


What does it mean?

- Diabetes technology research community intensively focused on AI
- Accurate **predictive algorithms** enable preventive actions → avoiding/mitigating onset of critical events

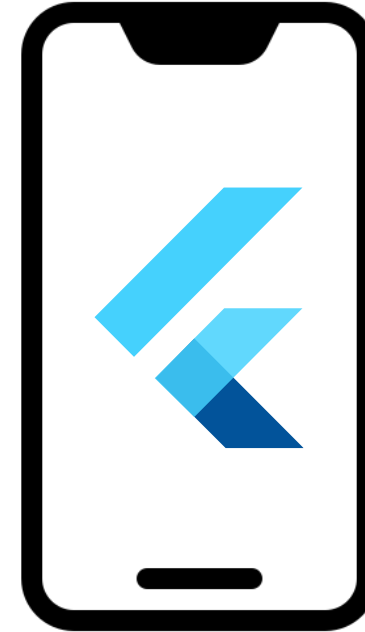


Aim



Part 1

Build a 30-minute glucose prediction model



Part 2

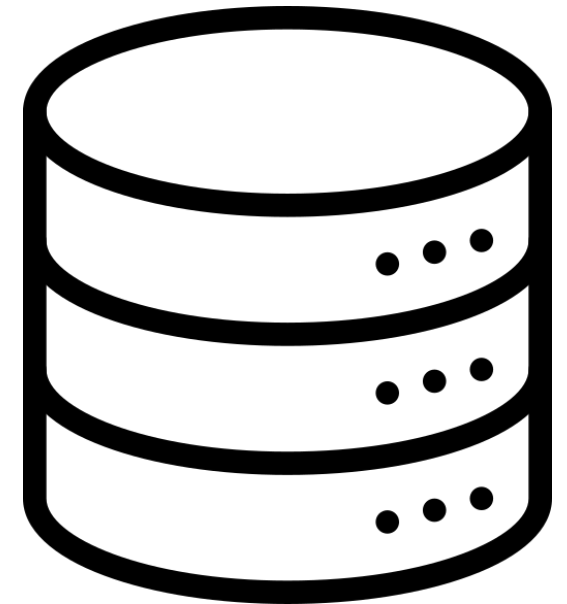
Integrate the model in a Flutter app

Aim

- Build a 30-minute glucose prediction model
 - Predict future CGM values using current CGM and Rate of Change
 - Train on real-world clinical data (ReplaceBG dataset)
- Learn the full Python ML pipeline
 - Data loading & preprocessing
 - Feature engineering
 - Model training, evaluation, and export
- Prepare for Flutter integration (Part 2)
 - Export model as a portable JSON file
 - Re-implement the prediction formula in Dart

The Dataset – ReplaceBG

- We will use data coming from the ReplaceBG clinical trial (<https://pmc.ncbi.nlm.nih.gov/articles/PMC5864100/>)
 - Studied whether CGM can replace blood glucose for insulin dosing decisions
 - Conducted by the T1D Exchange consortium
 - Available freely thanks to the MetaboNet initiative: <https://metabo-net.org/data>
- Dataset characteristics
 - ~226 Type 1 Diabetes patients
 - CGM readings sampled every 5 minutes
 - Columns: date (timestamp), CGM (mg/dL), id (patient)
- Available as a Parquet file
 - Efficient columnar binary format
 - Loaded with pandas + pyarrow
 - `df = pd.read_parquet('data/ReplaceBG.parquet')`



Setting Up

➤ Requirements

- Python 3.10+
- numpy, pandas, scikit-learn, pyarrow
- I provided a requirements file for convenience (`pip install -r requirements.txt`)

➤ Project structure

- `predict_glucose/`
- `data/ReplaceBG.parquet` # input data
- `main.py` # training script
- `model/cgm_forecast.json` # exported model
- `requirements.txt`

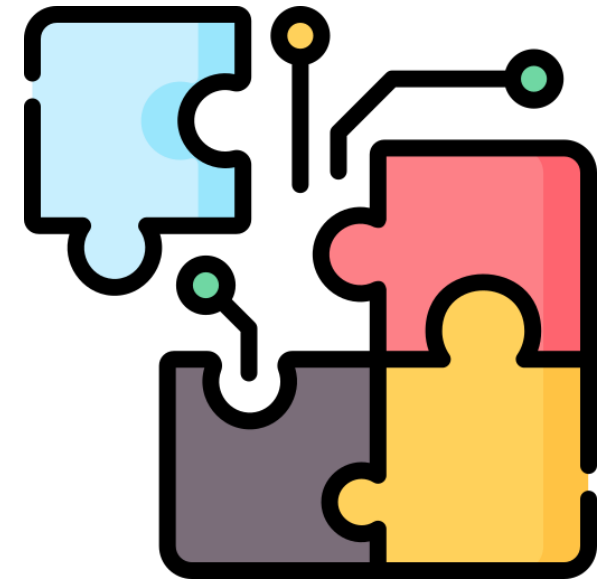
Step 1: Preparing the Data

- Load and filter
 - Keep only CGM, date, id columns
 - Sort each patient's readings by timestamp
- Patient-level train/test split
 - 80% of patients → training set
 - 20% of patients → test set
 - Avoids data leakage across patients
- Build the target vector Y
 - Shift CGM by 6 samples = 30 minutes ahead
 - $Y[t] = \text{CGM}[t + 6]$
 - Drop rows with NaN (edges of the time series)



Step 2: Feature Extraction

- Feature 1 – Current CGM value
 - Most informative single predictor of near-future glucose
 - Captures the current glucose level
- Feature 2 – Rate of Change (ROC)
 - Glucose velocity over the last 15 minutes (3 samples)
 - $ROC[t] = (CGM[t] - CGM[t-3]) / 15$ # mg/dL/min
 - Captures the glucose trend (rising, stable, falling)
- Final feature matrix X
 - $X = [CGM, ROC] \rightarrow \text{shape: } (N_samples, 2)$
 - Simple yet physiologically meaningful
 - Easy to compute in real-time on a mobile device



Step 3: Model selection and training pipeline

- sklearn Pipeline
 - StandardScaler: zero-mean, unit-variance normalization
 - LinearRegression: learns weights for CGM and ROC
 - `model = make_pipeline(StandardScaler(), LinearRegression())`
- Why linear regression?
 - Simple and interpretable
 - Prediction formula can be trivially re-implemented in Dart/Flutter
 - Fast inference: a single dot product
- Training
 - `model.fit(X_train, Y_train)`
 - Fits scaler parameters and regression coefficients on training patients

Step 4: Learning the model

➤ Learned prediction formula

- $Y = 58.40 * (CGM - 159.70)/64.55 + 11.84 * (ROC + 0.001)/0.952 + 159.66$

$$Y = 0.905 * CGM + 12.44 * ROC + 15.21$$

- ROC coefficient is positive: rising glucose → higher future value

How about performance?

- Evaluation on held-out test patients
 - `y_pred = model.predict(X_test)`
 - RMSE – Root Mean Squared Error (mg/dL)
 - R^2 – Coefficient of Determination
- Interpreting the results
 - RMSE ~ 20 mg/dL is clinically acceptable for 30-min horizon
 - R^2 close to 1.0 indicates strong predictive power
 - Linear model is surprisingly competitive as a baseline



Exporting the Model

- Export as JSON – language-agnostic format
 - No sklearn dependency needed at inference time
 - Can be bundled directly in Flutter app assets
- JSON structure
 - `feature_names`: ["CGM", "ROC"]
 - `mean`: [159.70, -0.0008]
 - `scale`: [64.55, 0.9516]
 - `coef`: [58.40, 11.84]
 - `intercept`: 159.66
- In Part 2 – Flutter integration
 - Load JSON from assets at app startup
 - Implement StandardScaler + dot product in Dart
 - Run real-time glucose predictions from live CGM stream

